Paper 3 — Escaping Saddle Points

Yann N. Dauphin, Razvan Pascanu, Çağlar Gülçehre, Kyunghyun Cho, Surya Ganguli, Yoshua Bengio (NeurIPS 2014)

Identifying and Attacking the Saddle Point Problem in High-Dimensional Non-Convex Optimization

Papers Master: `dauphin2014identifying` | G / D / E; F + B; **Must read (recommended)** | Geometry, Dynamics, Escape methods

The paper in one minute

Question: Why do high-dimensional neural-network loss surfaces produce long training plateaus, even when poor local minima may not be the main problem? **Model/Method:** Analyze Hessian eigenvalues and critical-point indices; compare GD with Newton near saddles; propose *saddle-free Newton* (SFN). **Finding:** Weak negative curvature can make GD escape slowly, while Newton can move *toward* saddles. SFN rescales the gradient using absolute Hessian eigenvalues and improves optimization in the tested networks. **Why we care:** Explains how landscape geometry motivates a concrete saddle-escape intervention.

1. Start with curvature, not the algorithm

A **critical point** has $\nabla f(\theta^*) = 0$. The **Hessian** $H = \nabla^2 f(\theta^*)$ tells us how the loss bends along each of its eigenvector directions: positive eigenvalue = bends up; negative eigenvalue = bends down. A *strict saddle* has at least one negative Hessian eigenvalue.

Take a two-dimensional teaching example (*not* an experimental model from the paper):

$$\boxed{f(x,y) = \tfrac{1}{2}x^2 - \tfrac{\varepsilon}{2}y^2, \qquad \varepsilon = 0.01.}$$

The gradient and Hessian are

$$\nabla f(x,y) = \begin{bmatrix} x \\ -\varepsilon y \end{bmatrix}, \qquad H = \begin{bmatrix} 1 & 0 \\ 0 & -\varepsilon \end{bmatrix}.$$

The origin is a saddle: it is a minimum along x but a maximum along y. Since $-\varepsilon = -0.01$ is close to zero, the downward direction is **almost flat**.

Why this matters in ML

A small gradient is not proof of being near a minimum. Along a nearly flat negative-curvature direction, the gradient can be tiny *even though moving away lowers the loss*. This is the key mechanism behind the paper's "saddle plateau" argument.

2. Why high dimensions change the picture

The paper draws on statistical-physics models of random Gaussian landscapes. It studies the **index** α of a critical point, the fraction of Hessian eigenvalues that are negative:

$$\alpha = \frac{\#\{\lambda_i(H) < 0\}}{\text{number of Hessian eigenvalues}}.$$

In those models, higher-error critical points tend to have a larger α; local minima have $\alpha = 0$. In small neural networks on downsampled MNIST/CIFAR-10, the authors observe a similar *qualitative* index–error trend (paper Fig. 1).

Important precision

The statistical-physics conclusions concern particular **random Gaussian landscapes**. Their transfer to trained neural networks is a hypothesis supported by experiments, **not** a universal theorem that every neural-network local minimum is good or every plateau is a saddle.

3. Why ordinary GD can be slow and Newton can be worse

Near a non-degenerate saddle, a local quadratic model separates motion into eigenvector directions. For one direction with eigenvalue λ and coordinate v measured from the saddle:

$$\nabla_v f \approx \lambda v, \qquad \underbrace{v^+_{\text{GD}} = (1-\eta\lambda)v}_{\text{gradient descent}}, \qquad \underbrace{v^+_{\text{Newton}} = 0}_{\text{quadratic model; nonsingular } H}.$$

For $\lambda < 0$, GD has factor $(1+\eta|\lambda|) > 1$: it moves *away*, but very slowly when $|\lambda|$ is tiny. Newton divides by the *signed* curvature and jumps *toward* the stationary point, even when that point is a saddle.

4. Worked example: one update, three different behaviors

Start at $(x_0, y_0) = (0, 1)$ on the earlier loss, use $\eta = 1$, and assume we can use the exact nonsingular Hessian. At that point,

$$\nabla f(0,1) = \begin{bmatrix} 0 \\ -0.01 \end{bmatrix}, \quad H^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & -100 \end{bmatrix}, \quad |H|^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 100 \end{bmatrix}.$$

Step 1 — Gradient descent. Follow the negative gradient:

$$(0,1) - \begin{bmatrix} 0 \\ -0.01 \end{bmatrix} = \boxed{(0, 1.01)}.$$

It escapes in the correct direction, but moves only 0.01 because negative curvature is weak.

Step 2 — Standard Newton. Rescale using *signed* eigenvalues:

$$(0,1) - H^{-1}\begin{bmatrix} 0 \\ -0.01 \end{bmatrix} = (0,1) - \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \boxed{(0,0)}.$$

Newton has landed exactly *on* the saddle for this quadratic example.

Step 3 — Saddle-free Newton. Rescale using the *magnitude* of curvature:

$$(0,1) - |H|^{-1}\begin{bmatrix} 0 \\ -0.01 \end{bmatrix} = (0,1) - \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \boxed{(0,2)}.$$

Here $f(0,1) = -0.005$, $f(0,1.01) \approx -0.00510$, and $f(0,2) = -0.020$.

Interpret the result, not just the numbers

GD goes downhill but slowly. Newton mistakenly heads to the saddle. SFN preserves the downhill direction *and* amplifies movement along weak negative curvature. This is a local quadratic teaching example; it is **not** a guarantee that full SFN always takes safe or faster steps on arbitrary losses.

5. What exactly does "absolute Hessian" mean?

For a symmetric Hessian with eigendecomposition $H = Q\,\mathrm{diag}(\lambda_1, \ldots, \lambda_d)Q^\top$, define

$$\boxed{|H| = Q\,\mathrm{diag}(|\lambda_1|, \ldots, |\lambda_d|)Q^\top}, \qquad \boxed{\Delta\theta_{\text{SFN}} = -\eta\,|H|^{-1}\nabla f(\theta)}.$$

This means *absolute eigenvalues*, **not** elementwise absolute values of entries. Along each eigenvector, SFN scales the gradient by $1/|\lambda_i|$. Its sign remains a descent direction when the inverse is well defined.

Critical technical caveat

Near-zero or zero eigenvalues make the ideal inverse ill-conditioned or undefined. The actual implementation uses **damping** and a low-dimensional approximation. Also, at an *exact* stationary point $\nabla f = 0$, the basic SFN update is zero; it does not magically move from every exact saddle.

6. Where does SFN come from? The trust-region intuition

The authors minimize a **first-order** approximation $f(\theta) + g^\top s$ while limiting how far they trust it. The difference between first- and second-order approximations is $\frac{1}{2} s^\top H s$. Their key bound is

$$\left| s^\top H s \right| \leq s^\top |H| s.$$

This motivates the *generalized trust-region* constraint $s^\top |H| s \leq \Delta$. Optimizing the linearized loss inside that region yields a direction proportional to $-|H|^{-1} g$ (the scalar factor is absorbed into the step size). This is why the paper argues for SFN rather than merely proposing an eigenvalue-sign trick (Sec. 5–6, Eq. 4–5).

7. How do they make the method practical?

Computing and diagonalizing a full Hessian is prohibitive for large models. The paper therefore constructs a **Krylov subspace** using Lanczos iterations, restricts the objective to that lower-dimensional subspace, and applies damped SFN there (Algorithm 1). It is an *approximation*: it may not capture every important curvature direction and still has substantial computational cost.

8. Findings, contributions, and limitations

What they tested. A small MLP on downsampled 10×10 MNIST/CIFAR-10 (comparing minibatch SGD, damped Newton, SFN); a seven-hidden-layer MNIST autoencoder; and an RNN on Penn Treebank. For the autoencoder, SGD plateaued around MSE 1.0 before SFN continuation, eventually reaching MSE 0.57; the paper compares this with a previously reported Hessian-free MSE 0.69 (Sec. 7).

Core contributions. (1) A geometric case for saddles as a high-dimensional optimization obstacle; (2) empirical measurements linking critical-point index with error; (3) an explanation of GD/Newton behavior near saddles; (4) a trust-region derivation and approximate implementation of saddle-free Newton.

Assumptions and limitations. Local eigenvalue analysis uses a smooth, approximately quadratic neighborhood and, for the clean Newton expressions, a nonsingular Hessian. The random-landscape theory does not prove the same statistics for arbitrary neural nets. SFN results are experimental, without a general finite-time escape or global-optimality theorem. The large-model trials *continue* from stalled SGD rather than proving that every plateau was a saddle; accuracy and computational efficiency should not be conflated.

Connection to Saxe and Lee — useful for our survey

Saxe (2014) explains why coupled weight dynamics and initialization can create plateaus, even in deep *linear* networks. **Dauphin (2014)** highlights weak-curvature saddle plateaus and proposes a Hessian-based intervention. **Lee (2016)** proves that GD avoids *convergence* to strict saddles almost surely under assumptions. There is no contradiction: *asymptotic avoidance* is different from *fast finite-time escape*.

9. Quick reference

Concept	Meaning / key expression
Strict saddle	Stationary point with at least one negative Hessian eigenvalue
Index α	Fraction of Hessian eigenvalues below zero
GD in eigen-direction	$v^+ = (1 - \eta\lambda)v$; weak negative curvature $\Rightarrow$ slow repulsion
Newton	$-H^{-1}g$; may attract iterates toward saddles
Saddle-free Newton	$-\lvert H\rvert^{-1}g$; same as Newton for positive-definite H
Absolute Hessian	$Q\,\mathrm{diag}(\lvert\lambda_i\rvert)Q^\top$, not elementwise absolute value
Scaling trick	Lanczos/Krylov subspace with damping; approximation, not free

One-paragraph note for Papers Master. High-dimensional saddles and nearly flat negative-curvature directions can cause slow training. Dauphin et al. explain why Newton can be attracted to saddles and derive SFN using $|H|^{-1}$ to preserve descent directions while rescaling weak curvature. Experiments favor SFN in several tested networks, but the random-landscape argument is not universal and scalable SFN remains approximate.

Source: Dauphin et al. (2014), NeurIPS paper, Sections 2–7, Figures 1–4, Algorithm 1. **Read the original PDF**. Scalar example and wording are pedagogical adaptations.